Confidence-Based Document Routing with Azure Content Understanding and Jev

Use case study, reference implementation, October 2026

Lovin Johnson Maxwell · Senior .NET/Azure Engineer & Solution Architect, Qatar · Document AI, ERPNext/Frappe, SAP integration, LLM workflows

Summary

doc-ai-jev-router is a TypeScript reference implementation that decides which business documents are safe to post straight into an ERP and which should go to a person. It extracts content with Azure AI Content Understanding, asks TypeSafe AI's Jev decision model three typed questions about each document, and applies an explicit threshold policy to route each one to autoPost or humanReview with a stated reason. The demo builds, tests and runs end to end in mock mode with five sample documents; no live Azure or Jev call has been made yet, so every number in this study is an illustrative mock value. The point of the work is the design: a routing decision that is typed, gated question by question, and auditable.

The problem

In accounts payable and ERP intake, extraction is not the decision. A model can read an invoice number, line items and a total and still leave the real question open: should this document be posted without a human looking at it?

Teams usually settle on one of two defaults. Auto-posting everything is fast but risky, because a misclassified document or a total that does not add up goes straight into the ledger. Reviewing everything is safe but slow, and it wastes reviewer time on documents that were never in doubt.

The useful question sits between those defaults: **which documents are safe to post straight through, and why?**

Answering it needs a decision that can be checked, explained and tuned, not just more extracted fields.

The approach

Pipeline

Each document passes through four stages. Extraction produces content; the decision layer turns that content into typed answers with confidence; a policy compares those answers against thresholds; and the router sends the document to one of two destinations.



Figure 1. Routing pipeline in doc-ai-jev-router.

Why typed decisions instead of free-text LLM output

A free-text answer such as "this looks like a valid invoice" is hard to act on. It has to be parsed, it can be phrased differently on every call, and it does not say how sure the model is. Jev answers each question with a choice or a

score, plus a confidence and a probability distribution over the allowed options. That gives the policy something it can compare against a number and log.

- **Bounded outputs.** Answers are constrained to a fixed set (for example consistent, inconsistent, cannot_tell), so downstream code never guesses at intent.
- **Measurable certainty.** Confidence and probabilities are first-class fields, so thresholds can be set, measured and tuned per document type once labelled data exists.
- **Auditability.** Every route carries the reasons that produced it, which makes the decision explainable to a reviewer or an auditor.

Whether Jev's confidences are well calibrated for a given document set is something to verify on labelled samples; the design makes that check straightforward rather than assuming it.

Why "review" is a valid answer, not an error

The document-type question deliberately includes review and other as options. When a scan is unreadable or a document does not fit the expected types, the correct outcome is to hand it to a person, and the system treats that as a normal route with a reason attached, not as an exception. A router that is allowed to say "I should not decide this" is safer than one forced to pick a type.

Implementation

Stack

- **Runtime:** Node 20, TypeScript command-line application.
- **Extraction:** @azure/ai-content-understanding 1.1.0 (API 2025-11-01), using ContentUnderstandingClient.analyze / analyzeBinary with the prebuilt-documentSearch analyzer. Configured through CONTENTUNDERSTANDING_ENDPOINT and CONTENTUNDERSTANDING_KEY.
- **Decision:** @typesafe-ai/sdk 0.6.0, calling TypeSafeClient.systemOne({ state, questions }) with choice and score questions. Each answer carries choice, confidence and probabilities. Model jev-latest (jev-1.13.0), text-only, 64k context.
- **Output:** autoPost goes to an ERP stub that only logs; nothing is sent to an ERP. humanReview records the reasons.

The three Jev questions and thresholds

Question	Type	Allowed answers	Gate to auto-post
Document type	choice	invoice, purchase_order, delivery_note, other, review	Not other/review; confidence ≥ 0.85
Totals consistency	choice	consistent, inconsistent, cannot_tell	consistent; confidence ≥ 0.85
Readiness	score	numeric score (levels 0, 1, 2 in demo output)	score ≥ 1.5; confidence ≥ 0.80

Table 1. Jev questions and the illustrative thresholds used in the demo. Thresholds need tuning on labelled documents.

Project layout

```
src/
  extract.ts   Azure Content Understanding extraction (or fixture)
  decide.ts    Jev typed questions (or scripted mock)
  route.ts     threshold policy -> autoPost | humanReview + reasons
  erp.ts       ERP auto-post stub (logs only)
  config.ts    thresholds and environment
  types.ts     shared types
  index.ts     CLI entry point
fixtures/      5 sample documents
.env.example   endpoint and key placeholders
```

Testing

13 vitest tests cover the routing policy: which combinations of type, consistency and readiness clear the gates and which are held, and what reasons are attached. Build, test and demo all pass in mock mode.

Walkthrough of the five sample documents

The demo runs five fixture documents through the pipeline. Extraction uses fixtures, and the Jev step is replaced by a mock decider whose confidences are hand-scripted in src/decide.ts. The table shows how the policy behaves, not how Jev performs.

Document	Type	Type conf.	Consistency	Cons. conf.	Readiness (conf.)	Route	Reason
clean-invoice	invoice	0.91	consistent	0.90	1.86 (0.91)	autoPost	Cleared all thresholds
purchase-order	purchase_order	0.90	consistent	0.87	1.74 (0.88)	autoPost	Cleared all thresholds
delivery-note	delivery_note	0.88	consistent	0.86	1.70 (0.86)	autoPost	Cleared all thresholds
ambiguous-scan	other	0.37	cannot_tell	0.48	0.42 (0.55)	humanReview	Type is other; totals cannot be checked; readiness low
inconsistent-invoice	invoice	0.92	inconsistent	0.86	1.80 (0.90)	humanReview	Totals are inconsistent

Table 2. Mock run: confidences are scripted for illustration, not Jev output. Thresholds: choice confidence ≥ 0.85, readiness score ≥ 1.5, readiness confidence ≥ 0.80.

> **Key insight.** The inconsistent invoice is the most confidently typed document in the set (type confidence 0.92), and its readiness score of 1.80 also clears the bar. It is still held, because the line amounts plus tax do not equal the printed total, and totals consistency is its own question with its own gate. One confident answer is not permission to auto-post; each question gates separately.

The ambiguous scan shows the other side: a skewed, unreadable document that the policy holds for four separate reasons (type is other, totals cannot be checked, readiness 0.42 below 1.5, readiness confidence 0.55 below 0.80). The reviewer sees all of them, not just a generic failure.

Where it fits

The router is designed to sit between document intake and ERP posting. The intended integration points below are future work; neither has been built in this demo.

- **ERPNext / Frappe purchase invoice intake.** Replace the ERP stub with creation of a Purchase Invoice in ERPNext/Frappe for documents that clear every gate, and a review queue entry for those that do not.
- **SAP posting via RFC/BAPI.** Post cleared documents to SAP through RFC/BAPI calls, keeping the same typed decision and reasons as the audit trail.
- **Per-type tuning.** Thresholds should be tuned per document type on labelled samples rather than shared globally; an invoice and a delivery note carry different risk.
- **Operating metrics.** The two numbers that tell you whether the router is earning its place are the straight-through rate (share auto-posted) and the review rate (share sent to a person), tracked alongside reviewer corrections.

Limitations and next steps

- **Not run live yet.** No live Azure or Jev call has been made; all confidences shown are scripted mock values, not model output or calibration evidence.
- **Jev is external and text-only.** Jev is a text-only model and a third-party service, not part of Azure. Extracted content leaves the Azure boundary when it is sent for a decision.
- **Thresholds are illustrative.** 0.85 / 1.5 / 0.80 are starting points for the demo, not tuned values.
- **Residency and security.** Data residency and a security review are required before any production use, especially for financial documents in regulated environments.

Next steps: run the pipeline live with Azure and Jev keys on a labelled sample set; tune thresholds per document type; and measure the straight-through rate, the review rate and the cost per document before considering an ERP integration.

References

- TypeSafe AI, Jev TypeScript guide: https://www.typesafeai.org/guides/jev-typescript
- TypeSafe AI, Jev models: https://docs.typesafe.ai/models
- Jev JavaScript SDK: https://www.jevcode.ai/en/sdk/javascript/
- Microsoft Learn, Azure AI Content Understanding overview: https://learn.microsoft.com/en-us/azure/ai-services/content-understanding/overview